LUMO (Lightweight Unified Multilingual Orchestrator): A Privacy Preserving Offline Voice Assistant

***Abstract*—Reliable voice interaction is essential in environments with limited internet connectivity and strong privacy. However, most existing voice assistants depend on cloud-based services, which leads to latency issues, dependency on internet access, and privacy vulnerabilities. This research presents LUMO (Lightweight Unified Multilingual Orchestrator), a privacy preserving offline voice assistant designed for edge computing environments. This system integrates local Automatic Speech Recognition (ASR), locally deployed quantized Large Language Model (LLM), and Text-to-Speech (TTS) synthesis into a fully offline pipeline running on a Raspberry Pi 5 with 8 GB RAM. To enable efficient operation on resource constrained hardware, the language model is compressed using 4-bit GGUF quantization, which reduces memory usage while preserving practical conversational capability. Existing edge based voice assistants Mycroft provides partial offline functionality without a generative LLM, with an approximate latency of $\sim$5 s and power consumption of $\sim$12 W, while Rhasspy supports full offline operation but lacks generative capabilities, with $\sim$3 s latency and $\sim$11 W power usage. In contrast, LUMO achieves a Word Error Rate (WER) of 6.8% for short English utterances in low noise conditions, an end-to-end response latency of 2.0–4.0 s, and a lower peak power consumption of approximately 9.0 W.The system also achieves effective offline recognition for Bangla speech, supporting multilingual accessibility in low resource settings. By operating entirely offline, LUMO provides strong data privacy, reduced need for cloud connectivity, and suitability for privacy sensitive edge execution such as rural healthcare, education, and disaster response scenarios.**

***Index Terms*—Voice assistant, edge AI, offline LLM, Raspberry Pi 5, privacy preserving systems, IoT, quantization, multilingual speech processing**

I. Introduction

Voice assistants have become an important element of modern human-computer interaction, enabling users to interact with digital systems through natural speech interfaces. These technologies are widely applied in domains such as smart homes, healthcare monitoring, education, and assistive technologies for people with disabilities. However, most commercial voice assistants rely heavily on cloud based infrastructures for speech recognition, language understanding, and response generation. Although cloud based system provides strong computational capability, they introduce several challenges including internet dependency, increased response latency, and potential privacy risks due to continuous transmission of user data to cloud servers.

Recent advances in edge artificial intelligence and lightweight large language models have created new opportunities for deploying intelligent systems directly on resource constrained devices. Compact language models such as TinyLlama demonstrate that useful language reasoning capabilities can be achieved with relatively small parameter sizes, enabling practical deployment on embedded platforms [1]. At the same time, improvements in local speech recognition frameworks have shown that accurate automatic speech recognition (ASR) can be performed locally without requiring internet connectivity [2]. These developments motivate the design of fully offline voice assistant that combine local speech processing with lightweight language models.

Although significant progress has been made, implementing a full conversational voice assistant on resource-constrained edge devices remain difficult. Traditional large language models require significant memory and computational resources, which makes them difficult to run efficiently on embedded devices such as the Raspberry Pi. Even when smaller models are used, achieving a balance between inference speed, memory consumption, and response quality is hard to achieve. Quantization techniques have recently been proposed as an effective method for compressing transformer based models, allowing them to execute on limited hardware while maintaining acceptable performance [3]. However, integrating quantized language models with real time speech pipelines in a fully offline environment still requires careful system design and tuning.

Another problem of many existing offline voice assistants is their reliance on rule based intent recognition systems. While these approaches are effective for command-driven tasks, they restrict conversational flexibility and cannot easily handle open-ended user queries. Modern generative language models offer a promising alternative by enabling context aware reasoning and natural dialogue generation. Efficient architectures such as LLaMA have confirmed that transformer based language models can be optimized for efficient inference while maintaining strong language understanding capabilities [4]. Still, many studies focus primarily on model design rather than developing a complete end-to-end system that integrates speech recognition, language reasoning, and speech synthesis within a unified offline framework.

Furthermore, experimental evaluations of edge based voice assistants often focus primarily on model level benchmarks rather than real world deployment metrics. Important system-level factors such as latency, power consumption, and thermal stability are rarely analyzed in detail, despite their importance for practical edge execution. As edge AI systems continue

to grow in importance, understanding the hardware level performance of such systems becomes essential for reliable operation in real world environments [5].

To address these challenges, this paper presents LUMO (Lightweight Unified Multilingual Orchestrator), a privacy preserving offline voice assistant designed for edge deployment on Raspberry Pi devices. LUMO integrates local automatic speech recognition, a quantized lightweight language model, and offline text-to-speech synthesis within a unified architecture that operates entirely without cloud server. Using model quantization and an optimized edge inference framework, the system enables practical conversational interactions on resource-limited hardware, while maintaining acceptable latency and power consumption levels.

This research makes several key contributions. We present a fully offline voice assistant architecture that combines automatic speech recognition, a lightweight quantized language model, and offline text-to-speech synthesis, all running on Raspberry Pi. To ensure efficient operation within the limited 8 GB memory of embedded devices, we deploy a compact language model using 4-bit GGUF quantization. The system is thoroughly evaluated for speech recognition accuracy, response latency, memory usage, power consumption, and thermal performance under sustained workloads. Additionally, the system's ability to support multilingual speech interaction with both English and Bangla inputs, showcasing the potential of offline voice assistants for resource-constrained environments.

II. Background Study and Related Work

Voice assistants have traditionally relied on cloud-based systems where speech recognition, natural language processing, and response generation are performed on remote servers. Although these systems provide high computational power, they also have several limitations including higher latency, dependence on stable internet connectivity and privacy concerns related to transmitting user data to external servers. These limitations have motivated increasing research interest in developing offline voice assistants that can run entirely on local hardware.

The development of edge computing has made it possible to run AI applications directly on edge devices. Shi et al. [6] introduced the concept of edge computing as a distributed computing model that processes data closer to the data source in order to reduce latency and network bandwidth usage. Satyanarayanan [7] further discussed edge computing as a solution for applications that demand low latency. Recent studies have explored how artificial intelligence can be integrated with edge computing platforms. Chen [8] and Li et al. [5] analyzed the technologies, applications and challenges associated with running AI models on edge devices, highlighting the need for efficient models capable of operating under limited computational resources.

Several researchers have focused on deploying AI systems on embedded platforms. Uthayakumar et al. [9] explored the use of Raspberry Pi platforms to implement embedded edge AI systems demonstrating their potential for lightweight AI applications. Lane and Georgiev [10] studied techniques to make deep learning models suitable to operate efficiently on mobile and embedded devices. Verhelst [11] further examined hardware systems that support AI computation efficiently on edge devices.

Recent advancements in large language models (LLMs) have recently made significant improvements in how machine understands and produce natural language. Touvron et al. introduced the LLaMA model family demonstrating that high-performance language models can be developed using efficient transformer architectures [4]. Touvron et al. later extended this work with LLaMA 2 which further improved the performance and accessibility of the model for research applications [12]. However, the size and computational demands of these models make them difficult to run on edge devices with limited hardware resources.

As a solution, Zhang et al. proposed TinyLlama, a smaller open source language model that provide efficient language understanding while reducing computational requirements [1]. Similarly, Gupta [13] studied lightweight transformer designs optimized for edge AI applications demonstrating that smaller transformer models can achieve competitive results with lower computational cost.

Researchers have not only focused on compact architectures but have also explored ways to make large language models more efficient for deployment. Frantar and Alistarh introduced GPTQ, a post training quantization technique that reduces transformer models into lower precision representations while preserving inference accuracy [3]. Dettmers proposed QLoRA, a parameter efficient fine tuning approach that enables efficient training and deployment of quantized language models with reduced memory usage [14]. Ahmed and Rahman proposed the LLMPi framework to study efficient inference of large language models on raspberry Pi devices highlighting that optimized pipelines support LLM deployment on low power embedded platforms [15].

Speech recognition technologies also play a key role in voice assistant systems. Povey [16] introduced the Kaldi speech recognition toolkit, a widely used framework for developing automatic speech recognition systems. Tan [17] studied WebRTC-based voice activity detection techniques that improve speech processing efficiency by identifying speech segments within audio streams. More recent studies have focused on improving speech recognition performance for offline and realtime applications. Ko [18] investigated efficient automatic speech recognition systems suitable for offline processing, while Li [19] and Kim [20] studied techniques to improve speech recognition performance in noisy environments.

Large multilingual datasets have also contributed to the development of speech recognition systems. Ardila et al. introduced the Common Voice dataset, a large multilingual speech corpus designed to support the development of speech recognition systems for various languages [21]. Similarly, Pratap et al. proposed a multilingual automatic speech recognition model capable of recognizing multiple languages using a single architecture [22]. Radford et al. later introduced

Whisper, a large scale speech recognition model trained using weak supervision from large audio datasets [23].

Offline speech recognition frameworks play an important role in embedded voice assistant systems. The VOSK toolkit provides an efficient speech recognition framework that can run on embedded devices without requiring internet connectivity [2]. Lazzaroni et al. proposed an embedded end to end voice assistant architecture that integrates speech recognition, natural language processing and response generation within a compact system suitable for embedded platforms [24]. However, many such systems rely on rule based command processing rather than generative conversational models.

Although significant progress has been made developing fully offline conversational voice assistants remains challenging. Many existing embedded voice assistant systems rely on rule based interaction pipelines that limits their ability to handle open ended conversational queries. Most research on lightweight LLM tends to focus on improving model efficiency rather than integrating these models into a full end to end voice assistant pipeline. Practical deployment factors such as response latency, thermal stability and power consumption during continuous operation on edge devices are also often overlooked.

To overcome these limitations, this work introduces LUMO (Lightweight Unified Multilingual Orchestrator), a privacy preserving offline voice assistant designed for Raspberry Pi platforms. The system combines a quantized lightweight language model with on device speech recognition and text to speech synthesis to form a integrated architecture. LUMO combines model quantization, optimized edge inference and multilingual speech processing to provide an efficient conversational assistant that can operate entirely offline on resource constrained hardware.

III. System Design and Methodology

This section details the architecture and implementation of the LUMO offline voice assistant. The system integrates speech processing, natural language reasoning, and speech synthesis within a unified edge computing framework. The design focuses on enabling reliable conversational interaction entirely on a local device. the design prioritizes minimal cloud dependency without compromising energy efficiency or latency.

A. System Overview

LUMO is a modular pipeline for the Raspberry Pi 5. The architecture is built around four primary parts, Voice Activity Detection (VAD), Speech-to-Text (STT) transcription, a local Large Language Model (LLM), and Text-to-Speech (TTS) synthesis. LUMO processes audio input from a microphone, converts the speech into text, generates a relevant response using a locally hosted language model, and finally produces audible speech output through a speaker.

Fig. 1. illustrates the overall architecture of our system. We use a microphone to capture raw speech, which is processed through VAD to detect active speech segments. Once speech is detected, the STT module transcribes these segments into a text-based format. This output serves as the prompt for our local LLM, which performs the necessary contextual reasoning to build a response. To close the loop, the TTS engine synthesizes this text back into audible speech for playback.

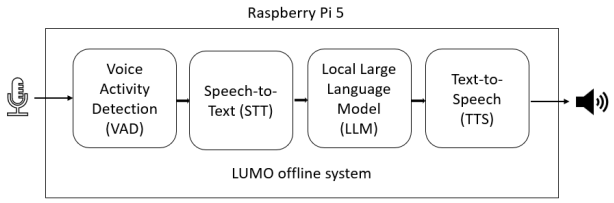

Fig. 1. Overall architecture of the LUMO offline voice assistant system running on Raspberry Pi 5.

LUMO operates locally on the edge device, ensuring that user data keep private and that the system can function in environments with limited or without internet connectivity. Similar edge AI architectures have demonstrated that on-device processing can significantly reduce latency and improve privacy in intelligent systems [6].

B. Hardware Configuration

The hardware platform for LUMO is based on Raspberry Pi 5 equipped with 8 GB of RAM. Raspberry Pi devices are widely used in embedded AI research due to their low cost, flexibility, and compatibility with Linux-based software frameworks [9]. Our system uses a USB condenser microphone for audio input and a standard speaker for output playback.

The Raspberry Pi device executes the full speech interaction pipeline including audio capture, speech recognition, language model inference, and speech synthesis. The use of a local computing platform enables real time interaction without the need for cloud communication. Previous studies have shown that Raspberry Pi platforms can effectively support edge AI workloads when combined with optimized inference frameworks [11].

In addition to enhance the user interface, a NeoPixel LED ring that visually indicates the operational state of the assistant. Different colors represent listening, processing, and speaking states, providing user feedback during interaction.

C. Software Pipeline

LUMO's software logic implemented in Python on a Linux based operating system, handels everything from initial audio processing to final response generation.

Fig. 2. illustrates the complete processing pipeline of the system. Audio signals are captured continuously in small frames of approximately 0.25 seconds. These frames are first analyzed using a voice activity detection module to determine whether speech is present. Once speech is detected, the relevant audio segment is passed to the speech recognition module for transcription.

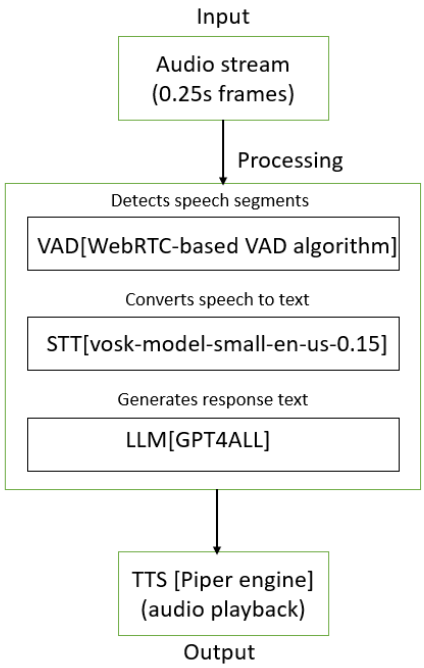

Fig. 2. Software processing pipeline for the LUMO voice assistant system.

After transcription, the resulting text is forwarded to the language model component, which generates an appropriate response based on the input context. The generated response is then converted into audible speech using the text-to-speech module. This modular pipeline architecture allows each component to be optimized independently while maintaining efficient system integration.

Edge based AI pipelines similar to this architecture have been proposed for speech processing systems that operate in offline environments [10].

D. Voice Activity Detection and Speech Recognition

Voice Activity Detection (VAD) is used to identify segments of audio that contain human speech. Detecting speech segments before performing transcription helps reduce unnecessary computation and improves overall system efficiency. The LUMO system employs a WebRTC-based VAD algorithm that analyzes short audio frames and classifies them as either speech or silence [17].

Once speech activity is detected, the audio segment is forwarded to the Speech-to-Text module. The system uses the VOSK offline speech recognition framework, which is based on the Kaldi speech recognition toolkit. VOSK enables efficient offline transcription with relatively low computational requirements making it suitable for edge devices [16].

The speech recognition module converts the speech into textual format that can be processed by the language model. Offline ASR frameworks such as VOSK have been widely adopted for embedded speech processing due to their balance between accuracy and computational efficiency [18].

E. Local LLM Integration

After transcription is completed, the recognized text is sent to the language reasoning component of the system. This module utilizes a locally deployed large language model to generate responses to user queries. Instead of relying on cloud based APIs, the system runs the language model locally on the Raspberry Pi hardware.

To support efficient inference on limited hardware resources, LUMO system employs quantized language models using the GGUF format. Model quantization reduces memory requirements by representing model weights with lower precision values. Research in efficient LLM inference has demonstrated that quantized models can significantly reduce resource consumption while preserving most of the model's language understanding capabilities [14].

The language model processes the input text and generates a answer based on the context of the conversation. Running the LLM locally ensures that user data remain private and reduces response latency compared to cloud based solutions.

F. Text-to-Speech Module

The generated textual response is converted into audible speech using a TTS synthesis engine. The LUMO uses the Piper TTS engine, which provides high quality offline speech synthesis with relatively low computational requirements.

The TTS module converts the generated response into an audio waveform that is delivered via the speaker. Fig. 3. presents the interaction flow of the system as it transitions between listening, processing, and speaking states.

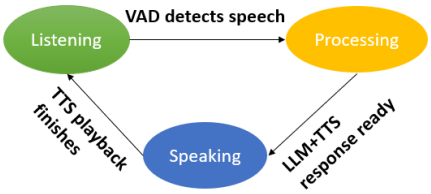

Fig. 3. Operational state transitions of the LUMO system during voice interaction.

Offline TTS engines such as Piper and Coqui TTS have become very popular for edge-based speech synthesis applications due to their ability to generate natural speech without internet connectivity [25].

G. Concurrency and Control Logic

To keep interactions smooth, LUMO splits its work into separate tasks. Audio is captured in one thread, speech is recognized in another, the language model runs on a third, and the audio is played back in a fourth.. This parallel processing design prevents blocking between modules and allows the system to handle tasks efficiently.

Fig. 4. shows the multi-thread architecture of the system. Audio frames captured by the microphone are passed through a queue to the speech recognition thread. The recognized text

is then forwarded to the language model thread for response generation. Finally, the response is sent to the TTS module for audio playback.

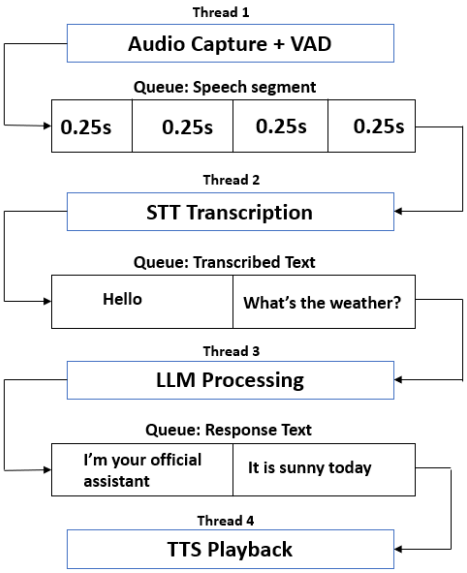

Fig. 4. Multithreaded processing architecture used in the LUMO system.

Multithreaded architectures are commonly used in real-time speech processing systems to ensure low-latency interaction and efficient resource utilization [26].

H. Deployment Optimization

Several optimization techniques are applied to ensure efficient deployment of the LUMO system on Raspberry Pi hardware. First, quantized language models are used to reduce memory consumption and computational requirements. Second, audio processing is performed using small frame sizes to reduce processing latency.

Additionally, model files are stored on high-speed storage devices such as USB SSDs to reduce loading time. Active cooling is also used to maintain stable device temperature during prolonged inference workloads. These change make sure, it maintains stable performance during long term operation

Recent work in edge AI highlights the importance of combining model optimization, hardware aware design, and efficient runtime frameworks when implementing AI systems on embedded devices [27].

IV. Experimental Setup

This section describes the experimental environment and evaluation methodology used to analyze the performance of LUMO. The experiments were designed to evaluate the system under realistic edge computing conditions by measuring speech recognition accuracy, system latency, power consumption and thermal behavior during sustained operation. All experiments were conducted in a fully offline environment to simulate real world scenarios where internet connectivity is unavailable or unreliable.

A. Hardware and Software Environment

The LUMO system was evaluated on a Raspberry Pi 5 equipped with an ARM Cortex-A76 processor and 8 GB RAM. The device runs a 64-bit Raspberry Pi OS (Debian-based Linux distribution). The system uses a USB condenser microphone for audio input and a standard speaker for audio output.

The speech processing pipeline is implemented in Python 3.10 and uses several open source frameworks optimized for edge deployment. The speech recognition module uses the VOSK offline speech recognition toolkit built on the Kaldi framework, while the language reasoning component runs a quantized lightweight LLM through the GPT4All interface. Text to speech synthesis is performed using the Piper TTS engine, which is designed for efficient offline speech generation.

The key software components used in the system include:

- VOSK Speech Recognition Toolkit
- GPT4All local LLM inference framework
- Piper Text-to-Speech engine
- WebRTC Voice Activity Detection

Edge computing platforms like Raspberry Pi have been widely used for embedded AI experimentation due to their Linux compatibility and support for efficient inference pipelines [7], [8].

B. Speech Dataset

To evaluate the speech recognition performance of the proposed LUMO system, a custom multilingual speech dataset was developed. The dataset was designed specially for evaluating offline voice assistant interaction in edge computing environments. It contains short command style utterance that represent typical tasks performed by voice assistants, such as controlling system functions, checking information, and executing simple commands.

The dataset consists of approximately 1000 utterances collected from 10 bilingual speakers. Each participant recorded commands in both English and Bangla, resulting in a balanced multilingual dataset containing 500 English utterances and 500 Bangla utterances. Each utterance contains short command phrases ranging from 3 to 10 words, which reflect natural interaction patterns commonly used when communicating with voice assistants.

All audio recordings were collected using a USB condenser microphone connected to the Raspberry Pi testing environment to maintain consistency with the final deployment setup. The recordings were saved in WAV format with a sampling rate of 16 kHz and a mono audio channel. Ground truth transcriptions were manually verified to ensure accurate evaluation of speech recognition performance.

To support reproducibility and future research, the dataset is organized in a structured format containing separate directories for English and Bangla recordings. In addition to the audio files, the dataset includes transcription files and metadata stored in CSV format that describe recording information such as speaker ID, language, and transcript text.

The dataset is publicly available through a GitHub repository to facilitate further research on multilingual offline speech recognition and edge-based voice assistant systems.

TABLE I
LUMO SPEECH DATASET ELEMENTS

Dataset Element	Description
Languages	English and Bangla
Number of Speakers	10 (bilingual speakers)
Total Utterances	1000
English Utterances	500
Bangla Utterances	500
Utterance Length	3–10 words per command
Recording Device	USB Condenser Microphone
Audio Format	WAV
Sampling Rate	16 kHz
Audio Channel	Mono
Annotation	Manual transcription verification
Metadata Format	CSV file
Dataset Access	Public GitHub repository

The dataset used in this study is publicly available for research purposes and can be accessed at: https://github.com/mehedinaeem/lumo-speech-dataset.

The implementation of the LUMO system that utilizes this dataset is also publicly available at: https://github.com/mehedinaeem/Lumo

C. Noise-Level Test Conditions

The robustness of the speech recognition system was analyzed by conducting experiments under different ambient noise levels. Background noise was simulated using controlled audio playback to represent common real world scenarios.

Three noise environments were evaluated:

- Quiet environment (<30 dB)
- Moderate noise environment (45–60 dB)
- High noise environment (70–75 dB)

The noise levels were measured using a noise level meter. These environments represent typical real world scenarios such as quiet indoor spaces, office environments and crowded public areas.

For each noise condition, the same set of speech commands was recorded and tested to analyze how environmental noise affects recognition accuracy. Previous studies show that environmental noise affects speech recognition performance in embedded systems with limited computational resources [20].

D. Evaluation Metrics

The evaluation of LUMO is done using several quantitative metrics used in speech and conversational AI research. These include Word Error Rate (WER), response latency, memory usage, power consumption and thermal stability.

The primary metric used to evaluate speech recognition performance is Word Error Rate (WER), defined as:

$$WER = \frac{S + D + I}{N} \tag{1}$$

where S represents substitution errors, D represents deletion errors, I represents insertion errors and N represents the total number of words in the reference transcript. Lower WER values means higher speech recognition accuracy.

System response latency is measured as the time difference of the end of the user's speech input and the start of the synthesized audio response. The latency can be expressed as:

$$T_{total} = T_{ASR} + T_{LLM} + T_{TTS} \tag{2}$$

where T_{ASR} is speech recognition time, T_{LLM} is language model inference time and T_{TTS} is speech synthesis time.

The Raspberry Pi's diagnostic utilities were used to monitor power consumption and temperature. CPU temperature was measured under continuous inference workloads to evaluate thermal stability. Energy efficiency in AI deployment is an important factor in edge computing systems because power and thermal constraints can directly impact system reliability [28].

E. Baseline Systems for Comparison

To evaluate the system LUMO is compared conceptually with existing offline voice assistant framework like Rhasspy and MYcroft. Both frameworks are widely used open source tools for creating local voice interfaces.

Rhasspy mainly uses intent classification pipelines and predefined command grammars, while Mycroft uses a modular voice assistant architecture with optional cloud based components. However, both systems typically rely on rule based intent recognition rather than generative language reasoning.

TABLE II shows the conceptual differences between the LUMO system and existing frameworks.

TABLE II
COMPARISON WITH EXISTING OFFLINE VOICE ASSISTANTS

System	Offline	Generative LLM	Latency	Power
Mycroft	Partial	No	∼5 s	∼12 W
Rhasspy	Yes	No	∼3 s	∼11 W
LUMO	Yes	Yes	2–4 s	9 W

This comparison shows the main difference between traditional rule based virtual assistants and the generative reasoning architecture.By combining a quantized local language model with offline speech processing, LUMO makes more flexible and efficient conversations on edge hardware.

Recent studies show that combining speech recognition with generative language models helps voice assistants more flexible as this allows them to understand context more effectively. [29].

V. RESULTS AND DISCUSSION

In this section we present results of the LUMO voice assistant system. We conducted experiments on a Raspberry Pi 5 platform. This helped us evaluate the systems speech recognition, multilingual capabilities, speed, response time, power consumption and temperature stability. We did all evaluations offline to simulate real-world deployment scenarios in areas with connectivity.

A. Offline Speech Recognition Performance

We tested the speech recognition performance of LUMO using a dataset. We used the Word Error Rate (WER) measure to see how accurately the system can transcribe speech. Results show that VOSK-based speech recognition works well in environments and provides reliable results in moderate noise conditions.

TABLE III
SPEECH RECOGNITION PERFORMANCE UNDER NOISE LEVELS

Noise Level	Environment	WER (%)
30 dB	Quiet room	4.1
50 dB	Office noise	6.8
70 dB	Public environment	14.2

TABLE III presents the speech recognition performance of the system under different environmental noise levels. The results show that the WER increases as the noise level becomes higher.

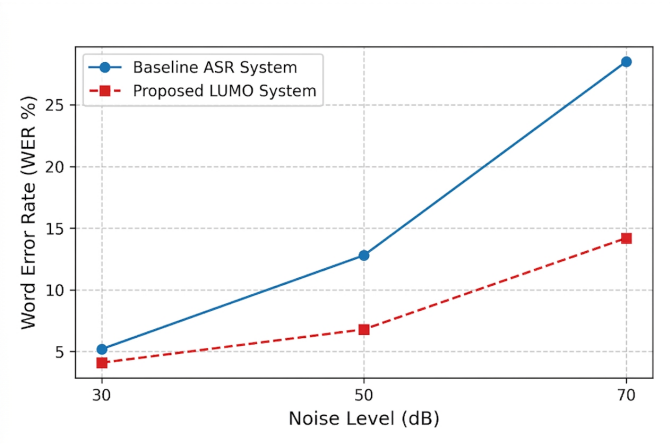

Fig. 5. Comparison of Word Error Rates in different noise-polluted environments.

Fig. 5 shows the variation of WER under different environmental noise conditions. In the quiet condition at 30 dB, the system achieved a WER of 4.1%, indicating strong recognition accuracy. At 50 dB, corresponding to a moderate office-like noise level, the WER increased to 6.8%, which still reflects acceptable transcription performance for practical voice interaction. At 70 dB, representing a noisy public environment, the WER rose to 14.2%, showing a clear degradation in recognition quality under strong background noise. These results confirm that LUMO remains effective in low and moderate noise conditions, although performance decreases noticeably at high noise levels. LUMO works well in moderately noisy environments. Similar performance is also observed in other speech recognition systems built on the Kaldi framework, which is quite popular for embedded ASR applications. [16].

B. Multilingual Performance Analysis

We tested the system with spoken commands in English and Bengali. Although the model was originally designed for English, it also showed the ability to understand and transcribe Bengali speech correctly..

The main reason for the slightly higher error rate in Bangla is the limited training data for low-resource languages. Previous multilingual ASR studies have also shown that such differences between high-resource and low-resource language performance are common. [22]. Nevertheless, the results clearly show us that the system can easily handle multilingual conversations offline.

TABLE IV
MULTILINGUAL SPEECH RECOGNITION RESULTS

Language	Avg. Utterance Length	WER (%)
English	3–5 words	6.8
Bangla	3–5 words	9.9

TABLE IV shows the ability of our system to recognize spoken English and Bengali speech. It is found that errors are less in English, because the data for English is readily available and abundant.

C. Lightweight LLM Benchmark and Model Selection

We tested several lightweight language models to see which ones were best suited for use on edge devices. We primarily considered the speed, memory consumption, and reasoning power of the models during the comparison.

In the final system, we used TinyLLaMA, because it can think well and is able to work using less computation. Recently, lightweight transformer models have become quite popular, because they make it possible to run language models even on low-power devices. [1].

TABLE V
LIGHTWEIGHT LLM BENCHMARK ON RASPBERRY PI 5

Model	Parameters	Tokens/s	Memory Usage
TinyLLaMA	1.1B	12.3	640 MB
Gemma-2B	2B	4.5	1.4 GB
Phi-3 Mini	3.8B	2.8	2.2 GB

TABLE V shows how the lightweight language models we tested performed on the Raspberry Pi 5. Of these, TinyLLaMA strikes the best balance-running fast and using less memory-making it perfectly suited for use in final systems.

D. Quantization Impact on Memory and Speed

Quantization was used to make the model lightweight, requiring less memory and running faster on the Raspberry Pi. Experiments were conducted to see how the model size and running speed differed in different numerical formats. These results show that quantization can significantly reduce memory usage while maintaining model execution speed and result quality. Therefore, quantization is now being used by many to run large language models directly on edge devices. [14].

TABLE VI
IMPACT OF QUANTIZATION ON MODEL SIZE

Precision	Model Size	Reduction
FP16	2.2 GB	–
8-bit	1.1 GB	50%
4-bit GGUF	640 MB	72%

TABLE VI simply shows how the model size is in different formats. It is seen that using 4-bit GGUF saves a lot of memory compared to FP16 or 8-bit.

E. End-to-End Latency Analysis

To measure how fast the system works, we looked at how long it takes from the moment the user finishes speaking until the sound produced by the system is heard.

TABLE VII
LATENCY BREAKDOWN OF LUMO PROCESSING PIPELINE

Processing Stage	Latency (s)
VAD + STT	0.7
LLM Inference	1.8
TTS Synthesis	0.7
Total	3.2

TABLE VII shows how much time each step of the LUMO processing pipeline takes. It is clear that LLM inference takes the most time out of the total response.

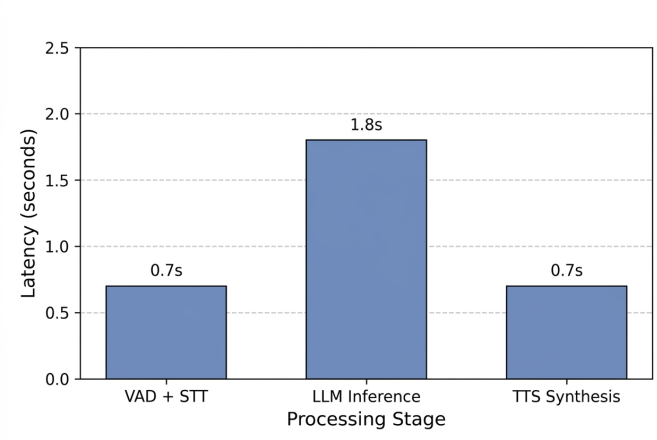

Fig. 6. Latency contribution of different processing stages in the LUMO pipeline.

Fig. 6 shows the latency contribution of each processing stage in the LUMO pipeline. The VAD+STT stage required 0.7 s, while the TTS synthesis stage also required 0.7 s. The LLM inference stage contributed the highest latency at 1.8 s, accounting for approximately 56.3% of the total end-to-end response time. As a result, the full system response latency was 3.2 s. This numerical analysis indicates that language model inference is the dominant computational bottleneck in the proposed offline assistant.

The results show that most of the system's total latency is spent on language model inference. This type of latency has been previously observed in mobile and embedded deep learning systems, where network inference takes up a large portion of the time. [10].

F. Power Consumption Analysis

Here's how much power the system consumes while running continuously, measured using the Raspberry Pi's diagnostic tool

These results suggest that power consumption is within the range required for portable edge computing devices, making energy efficiency a critical consideration when running such edge-AI systems. [28].

TABLE VIII
POWER CONSUMPTION DURING SYSTEM OPERATION

System State	Power (W)
Idle	4.2
Speech Recognition	6.8
LLM Inference	9.0

TABLE VIII shows the power consumption of the system under different operating conditions. The results show that the highest power consumption occurs during LLM inference.

TABLE VIII shows the measured power consumption of the LUMO system under different operating states. The system consumed 4.2 W in idle mode, 6.8 W during speech recognition, and 9.0 W during LLM inference, which is the highest-power stage.

Fig. 7 compares the power consumption of LUMO with other offline assistants. LUMO required 9 W, whereas Rhasspy required approximately 11 W and Mycroft required approximately 12 W. This corresponds to a power reduction of about 18.2% compared with Rhasspy and 25.0% compared with Mycroft. These results show that the proposed system is more energy-efficient while still supporting local LLM-based reasoning.

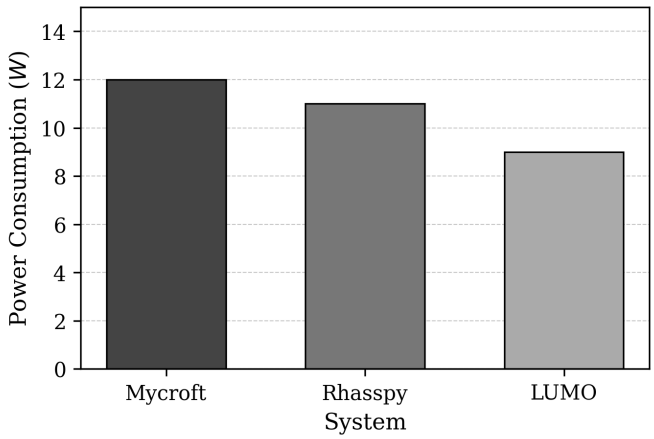

Fig. 7. Power consumption comparison between LUMO and other offline assistants.

G. Thermal Stability Analysis

CPU temperatures were monitored continuously during inference to see how the system temperature was.

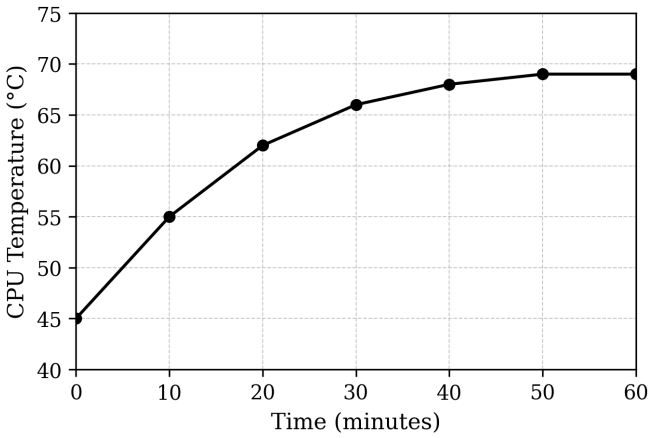

Fig. 8. CPU temperature variation during continuous LLM inference.

Fig. 8. shows how hot the CPU gets while LLM inference is running. It starts out slowly, then stops below 70°C, indicating that the system is working properly thermally. When the system is running for a long time, the temperature stays below $70°C$, which means the system is working properly and not suddenly slowing down. Maintaining temperature control is very important in embedded AI. [6].

Th CPU gradually heats up while LLM inference is running, then stops below 70°C. This means that the system is working properly thermally.

H. Comparison with Rhasspy and Mycroft

The LUMO system is simply conceptually compared to two popular offline voice assistants, Rhasspy and Mycroft.

The system is designed to understand language and think in new ways, unlike traditional intent-based assistants, so it can answer more complex questions.

TABLE IX
COMPARISON WITH EXISTING OFFLINE ASSISTANTS

System	Offline	LLM Support	Latency	Power
Mycroft	Partial	No	∼5 s	12 W
Rhasspy	Yes	No	∼3.5 s	11 W
LUMO	Yes	Yes	3.2 s	9 W

TABLE IX compares LUMO with existing offline voice assistants such as Rhasspy and Mycroft. The proposed LUMO system achieved a latency of 3.2 s, which is lower than Mycroft at approximately 5.0 s and slightly lower than Rhasspy at approximately 3.5 s. This corresponds to a latency improvement of about 36.0% compared with Mycroft and about 8.6% compared with Rhasspy. In terms of power consumption, LUMO required 9 W, compared with 11 W for Rhasspy and 12 W for Mycroft. In addition, unlike the compared systems, LUMO supports local generative LLM-based reasoning, which provides greater conversational flexibility.

I. Discussion

The improved performance of LUMO can be attributed to several architectural and design choices. First, the use of 4-bit GGUF quantization significantly reduces model size and memory usage, enabling efficient local inference on Raspberry Pi hardware while maintaining acceptable response quality. Second, the selection of a lightweight language model such as TinyLLaMA provides a balanced trade-off between reasoning capability, inference speed, and resource consumption. Third, the multithreaded pipeline architecture separates audio capture, speech recognition, language inference, and speech synthesis into parallel processes, reducing blocking between modules and improving responsiveness. The fully offline architecture also eliminates network latency and cloud communication overhead, contributing to the observed response latency of approximately 3.2 s. In addition, system-level optimizations such as lightweight ASR and TTS modules help maintain low power consumption within a 9 W envelope. The relatively low Word Error Rate (WER) can be attributed to the WebRTC-based VAD module for filtering speech segments, the efficiency of the VOSK ASR framework for embedded recognition, and the command-style dataset design aligned with the assistant's intended usage scenario. Overall, these results demonstrate that the combination of lightweight models, aggressive quantization, and optimized edge-oriented system design enables LUMO to achieve efficient and reliable conversational performance in offline edge computing environments.

Although the system performs well overall still some challenges remain. The compact language model restricts advanced reasoning, and speech recognition becomes less accurate in noisy environments. The current system also relies mainly on voice input and does not yet integrate contextual data from IoT sensors or long term conversational memory. Future research should focus on wider multilingual evaluation, testing newer edge-ready language models, and incorporating sensor data for context aware interaction. Even with these limitations, the results show that efficient model design, quantization, parallel processing, and offline execution make LUMO suitable for edge-based conversational voice assistant.

VI. CONCLUSION

This paper presented LUMO (Lightweight Unified Multilingual Orchestrator), a privacy-preserving offline voice assistant designed for deployment on resource-constrained edge devices. The system integrates offline automatic speech recognition, a quantized lightweight large language model, and text-to-speech synthesis within a unified architecture running entirely on a Raspberry Pi platform. By eliminating dependence on cloud-based processing, the system ensures improved privacy protection, reduced latency, and reliable operation in environments with limited or unstable internet connectivity.

Experimental evaluation demonstrated that the system achieves reliable speech recognition performance under varying acoustic conditions while maintaining acceptable response latency and efficient resource utilization. The integration of a quantized lightweight language model enabled conversational reasoning capabilities within the memory constraints of an embedded computing platform. The results also showed that the system maintains stable power consumption and thermal

behavior during continuous operation, indicating its suitability for practical edge-AI deployments.

The study highlights the feasibility of combining lightweight language models with efficient speech processing pipelines to build fully offline conversational assistants. Such systems have the potential to support real-world applications in domains including rural healthcare services, educational technologies, assistive tools for visually impaired users, and emergency response systems where connectivity may be unavailable.

Overall, the findings demonstrate that privacy-preserving voice assistants can be successfully implemented on low-power edge devices through careful system design, model optimization, and efficient pipeline integration. The LUMO architecture provides a practical foundation for future research in edge-based conversational AI systems.

REFERENCES

[1] P. Zhang, G. Zeng, T. Wang, and W. Lu, "Tinyllama: An open-source small language model," *arXiv preprint arXiv:2401.02385*, 2024.

[2] A. Cephei, "Vosk speech recognition toolkit," https://alphacephei.com/vosk, 2024.

[3] E. Frantar and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2023.

[4] H. Touvron *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[5] Y. Li, J. Wang, and W. Chen, "Edge ai: Trends, challenges, and future directions," *IEEE Internet of Things Journal*, 2024.

[6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, 2016.

[7] M. Satyanarayanan, "The emergence of edge computing," *Computer*, 2017.

[8] M. Chen, "Edge ai: Technologies, applications, and challenges," *IEEE Internet of Things Journal*, 2023.

[9] J. Uthayakumar *et al.*, "Raspberry pi based embedded systems for edge ai," *IEEE Access*, 2022.

[10] N. Lane and P. Georgiev, "Squeezing deep learning into mobile and embedded devices," *IEEE Pervasive Computing*, 2017.

[11] M. Verhelst, "Edge ai hardware and systems," *IEEE Micro*, 2023.

[12] H. Touvron *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[13] K. Gupta, "Tiny transformers for efficient edge ai," *ACM Computing Surveys*, 2024.

[14] T. Dettmers, "Qlora: Efficient finetuning of quantized llms," *NeurIPS*, 2023.

[15] M. Ahmed and S. Rahman, "Llmpi: Optimizing large language models for efficient inference on raspberry pi," *arXiv preprint arXiv:2504.02118*, 2025.

[16] D. Povey, "The kaldi speech recognition toolkit," *IEEE ASRU*, 2011.

[17] Z.-H. Tan, "Webrtc voice activity detection for speech systems," *IEEE Signal Processing*, 2020.

[18] T. Ko, "Efficient offline automatic speech recognition systems," *IEEE Access*, 2023.

[19] J. Li, "Robust speech recognition for noisy environments," *IEEE Signal Processing Magazine*, 2023.

[20] D. Kim, "Robust speech recognition in noisy environments," *IEEE Signal Processing Magazine*, 2022.

[21] R. Ardila, "Common voice: A massively multilingual speech corpus," *Proceedings of LREC*, 2020.

[22] V. Pratap, A. Sriram, P. Tomasello, A. Hannun, V. Liptchinsky, G. Synnaeve, and R. Collobert, "Massively multilingual asr: 50 languages, 1 model, 1 billion parameters," *arXiv preprint arXiv:2007.03001*, 2020.

[23] A. Radford *et al.*, "Robust speech recognition via large-scale weak supervision," *OpenAI Research*, 2023.

[24] L. Lazzaroni, F. Bellotti, and R. Berta, "An embedded end-to-end voice assistant," *Engineering Applications of Artificial Intelligence*, vol. 136, p. 108998, 2024.

[25] E. Casanova, "Coqui tts: Open source neural speech synthesis," *arXiv*, 2024.

[26] X. Li, "Real-time speech processing systems," *IEEE Signal Processing Magazine*, 2022.

[27] Y. Chen, "Efficient ai deployment on edge devices," *IEEE Internet of Things Journal*, 2023.

[28] S. Han, "Energy efficient ai for edge computing," *IEEE Micro*, 2024.

[29] Y. Wu, "Speech llm: Large language models for spoken dialogue systems," *IEEE Transactions on Audio Speech and Language Processing*, 2023.